

User Guide

How to run, sign in to, and use the Pont gateway and web console: one API key and one endpoint for OpenRouter and OpenCode Go.

UNIFIEDPROVIDERS LOCAL DOCKER EDITION OCTOBER 2026

GUIDE

Contents

01 **What Pont is** 3
- Two credentials 3
- Pont model IDs 4

02 **Accessing the service** 5
- URLs and ports 5
- Starting and stopping with Docker 5
- Configuration 6

03 **Signing in** 7

04 **Roles: Administrator and Member** 8

05 **Everyday tasks (all users)** 9
- Read the Overview dashboard 9
- Create or rotate your gateway key 9
- Find a model to use 9
- Review your request logs 10
- Try requests in the Endpoints explorer 10

06 **Administration (admins only)** 11
- Set provider keys 11
- Choose eligible models 11
- Manage users 11

07 **Using the API** 13
- Gateway API (LLM calls) 13
- Management API 15

08 **Troubleshooting and FAQ** 17

CHAPTER 01

What Pont is

Pont is a self-hosted LLM gateway with a web console. Your apps send requests to one OpenAI- and Anthropic-compatible endpoint. Pont checks the caller's personal key, sends the request to the right provider, and logs usage.

`Your app / SDK` → `Pont gateway :9880` → `OpenRouter · OpenCode Go`

GATEWAY
One endpoint, many providers
`/v1/chat/completions`, `/v1/responses` and `/v1/messages`. The model-name prefix chooses the provider.

KEYS
Personal gateway keys
Each user gets their own `pont_sk_…` key. Provider keys are stored centrally and never sent to clients.

CONTROL
Model allowlist
Admins choose which models the gateway accepts (*Eligible models*).

VISIBILITY
Logs and usage
Each completion is logged with app, model, tokens, latency and status. The dashboard shows a 24-hour summary.

Components

COMPONENT	CONTAINER	ROLE
Pont API	`pont-pont-api-1`	NestJS service on port `9880`. Hosts the gateway routes, the management API (`/api/…`) and health checks. Data is stored in SQLite at `/data/pont.db` on the Docker volume `pont_pont-data`.
Pont WebUI	`pont-pont-webui-1`	Static console served by nginx on port `9881`. It forwards `/api` and the gateway paths to the API container.

Two credentials: know which one to use

CREDENTIAL	HOW YOU GET IT	USED FOR
Session token	Sign in to the WebUI or call `POST /api/login`. It is valid for 24 hours.	The console and the management API (`/api/…`).

CREDENTIAL	HOW YOU GET IT	USED FOR
Gateway key `pont_sk_…`	Created when your account is created, or rotated on the *API key* page. The full key is shown **only once**.	LLM calls to `/v1`, `/openrouter/v1`, `/opencode-go/v1`, `/zen/go/v1`.

> **NOTE**
> The two credentials don't work in each other's place. A session token on `/v1/…` returns *"Invalid or missing Pont API key"*. A gateway key on `/api/…` returns *"Invalid or expired token"*.

Pont model IDs

Pont names every model as `<provider-prefix>-<model-slug>`, with an optional `-free` suffix for free-tier variants:

PREFIX	PROVIDER	HOW UPSTREAM IDS ARE TRANSLATED
`opencode-…`	OpenCode Go	The slug is passed through unchanged. Example: `opencode-deepseek-4-1-free`.
`openrouter-…`	OpenRouter	`vendor/model` becomes `vendor-model`, and `:free` becomes `-free`. Example: `meta-llama/x:free` → `openrouter-meta-llama-x-free`.

You don't need to work these IDs out yourself. Copy them from the **ID** column on the *Models* page or from `GET /v1/models`.

CHAPTER 02

Accessing the service

Pont runs in Docker on your machine as the Compose project `pont`. Both ports are published on all interfaces of the host.

URLs and ports

URL	WHAT IT IS
`http://127.0.0.1:9881`	**Web console.** Open this in your browser.
`http://127.0.0.1:9880`	**API and gateway**: `/v1`, `/openrouter/v1`, `/opencode-go/v1`, `/zen/go/v1`, `/api`, `/health`.
`http://127.0.0.1:9881/v1` …	The same gateway and `/health` routes, forwarded by nginx through the console's origin. The management API is under `/api` there too.
`http://127.0.0.1:9880/health`	Liveness check. It returns `{"ok":true,"service":"@pont/api"}`.

Console pages use hash routes, so you can bookmark them, for example `http://127.0.0.1:9881/#/models` or `#/logs`, `#/api-key`, `#/endpoints`, `#/providers`, `#/eligible-models`, `#/users`.

Starting and stopping with Docker

Run these commands in PowerShell from the repository root (`C:\Users\armin\GitHub\UnifiedProviders`):

START (BUILD IMAGES IF NEEDED)

```
docker compose up -d --build
```

CHECK STATUS AND FOLLOW THE API LOGS

```
docker compose ps
docker compose logs -f pont-api
```

STOP (DATA IS KEPT)

```
docker compose down
```

> **KEEP YOUR DATA**
>
> Users, keys, settings and logs are stored in the `pont_pont-data` volume. Never run `docker compose down -v`, because that deletes the volume.

The API container has a health check that polls `/health` every 10 seconds. The WebUI starts only after the API reports *healthy*. Both containers restart automatically (`restart: unless-stopped`).

Configuration

Compose reads these variables from your shell or from a `.env` file next to `docker-compose.yml`:

VARIABLE	PURPOSE
`PONT_ADMIN_JWT_SECRET`	Secret used to sign console session tokens. Set your own value, because the built-in default is a placeholder.
`OPENROUTER_API_KEY`	Fallback OpenRouter key. A key saved under *Providers* takes precedence.
`OPENCODE_GO_API_KEY`	Fallback OpenCode Go key. A key saved under *Providers* takes precedence.

The compose file also sets `PONT_PORT=9880`, `PONT_WEBUI_ORIGIN=http://127.0.0.1:9881` (the browser origin allowed by CORS) and `PONT_OPENVPN_SKIP=1`. Other variables the API reads: `PONT_DB_PATH`, `PONT_USER_AGENT`, `OPENCODE_GO_BASE_URL` and `PONT_OPENVPN_CONFIG`.

Optional VPN egress

Pont can send all upstream traffic through OpenVPN. In the shipped compose file, VPN is **turned off** (`PONT_OPENVPN_SKIP=1`). To turn it on, place a client profile in `openvpn/` (default name `client.ovpn`, plus `auth.txt` if the profile needs it) and stop skipping VPN. The container then waits up to 60 seconds for `tun0` and exits if the tunnel doesn't come up. The *Overview* page shows the tunnel state.

CHAPTER 03

Signing in

Each person has their own account. You sign in with a username and password, and you call the gateway with your own gateway key.

Accounts on this installation

USERNAME	ROLE	TYPICAL USE
`admin`	Administrator	Manages providers, models and users. Sees gateway-wide usage.
`warehouse-management`	Member	Calls the gateway with its own key. Sees only its own usage and logs.

> **PASSWORDS**
> This guide contains no passwords. Get your credentials from the administrator through a secure channel.

Sign in to the console

1. Open `http://127.0.0.1:9881`. If you aren't signed in, you are taken to the sign-in form.
2. Enter your **Username** and **Password**, then click **Sign in**.
3. The *Overview* page opens. The left sidebar shows the pages you can use. The bottom of the sidebar shows your name, username and role (*Administrator* or *Member*).
4. To finish, click **Sign out** at the bottom of the sidebar.

Good to know

- A failed sign-in always shows *"Invalid username or password"*. Disabled accounts can't sign in.
- Your session lasts **24 hours** and is kept only for the current browser tab. If you close the tab or the session expires, you return to the sign-in page.
- If an admin disables your account or removes your admin rights, the change applies to your next request, even if your session hasn't expired.
- The console has no self-service password change. Ask an administrator to set a new password (see Management API).

CHAPTER 04

Roles: Administrator and Member

Pont has two roles. Members use the gateway and see their own activity. Administrators also control providers, the model allowlist and accounts.

CAPABILITY	MEMBER	ADMINISTRATOR
Call the gateway with your own key	✓	✓
Overview: usage for the last 24 hours	Own	All users, plus a *By user* table
Models: browse the catalog with prices	✓	✓
API key: create or rotate *your own* key	✓	✓
Request logs	Own	All users, with a User column
Endpoints explorer	Gateway, Health and API routes	Also Admin routes
VPN status panel	✓	✓
Providers: set or clear upstream API keys	—	✓
Eligible models: edit the gateway allowlist	—	✓
Users: list, create, rotate keys	—	✓
Update, disable, delete users, reset passwords (API)	—	✓

Admin pages (*Providers*, *Eligible models*, *Users*) aren't shown in the sidebar for members. If a member opens them by URL, they see *"Not found"*. On the API side, admin routes return `401 Admin access required`.

CHAPTER 05

Everyday tasks

Step-by-step guides for the pages every user has: Overview, API key, Models, Request logs and Endpoints.

Read the Overview dashboard

1. Click **Overview** in the sidebar. This is also the first page after you sign in.
2. The cards show *Requests (24h)*, *Tokens (24h)*, *Avg latency*, *Error rate*, *All-time requests* and *All-time tokens*. Members see their own numbers. Admins see the whole gateway.
3. The **VPN** panel shows the tunnel state (*Connected*, *Skipped* or *Down*) and the public IP that Pont currently uses. Click **Refresh** to check again.
4. The **By app** and **By model** tables show requests and tokens. Admins also get a **By user** table.

Create or rotate your gateway key

1. Open **API key**. The page shows your current key prefix, for example `pont_sk_AbCd…`.
2. Click **Create / rotate key**. A new key appears **once**. Copy it now and store it safely.
3. Update your apps right away. **Your previous key stops working at once.**
4. The page also lists the three base URLs you can call. The browser remembers the new key so that the Endpoints explorer can use it.

> **LOST KEY?**
> Pont stores only a hash of your key, so nobody can recover it. Rotate it to get a new one.

Find a model to use

1. Open **Models**. It lists models from every provider that has an API key, with context length and upstream prices in USD per 1M tokens (input, input cache, output, output cache).
2. Use the filters: **Provider** (OpenCode Go by default, or *All providers*), **Free tier** (*All*, *Free only*, *Paid only*) and **Search** (model, owner or ID; press Enter).
3. Click a column header to sort it. Click it again to reverse the order.
4. Copy the value in the **ID** column. This is the `model` value you send to the gateway.

> **IMPORTANT**
> The Models page shows everything the providers offer. The gateway accepts only models that an admin has marked **eligible**. If a call fails with *"Model is not eligible for this gateway"*, ask an admin to enable that model.

Review your request logs

1. Open **Request logs**. Members see their own requests. Admins see all users, with an extra *User* column.
2. Each row shows *Time*, *App*, *Model*, *Prompt* (a short title from the request), *Tokens*, duration in *ms* and HTTP *Status*. Errors (400 and above) are highlighted.
3. Use **Previous** and **Next** to page through the logs, 50 entries per page.

The *App* column comes from the optional `x-yoda-client-app` header (see Gateway API). Requests without it are logged as `unknown`.

Try requests in the Endpoints explorer

1. Open **Endpoints**. Routes are grouped as *Gateway*, *Gateway (pinned provider)*, *Gateway (legacy)*, *Health* and *API*. Admins also see *Admin*.
2. Click a route. The **Try it** panel fills in the request URL and a sample JSON body.
3. Replace placeholders such as `:username` or `opencode-<model-slug>` with real values.
4. Choose **Authorization**: *Session token* (your sign-in), *Gateway key* (paste your `pont_sk_…` key) or *None*.
5. Click **Send**. The panel shows the status, duration, content type and the formatted response. `/api/events` is a live stream, so the explorer shows a 3-second sample.

> **CAREFUL**
>
> The explorer sends **real** requests. Admin routes such as user delete or provider key changes take effect immediately.

CHAPTER 06

Administration

Admin-only pages. Set them up in this order: provider keys first, then eligible models, then users.

Set provider keys

1. Open **Providers**. There is one card per provider: `openrouter` and `opencode-go`.
2. The badge shows where the active key comes from: `settings` (saved here), `env` (from the container environment) or `none`. A masked preview of the key is shown.
3. Paste a key into **API key** and click **Save**. A saved key always takes precedence over the environment variable.
4. To clear the saved key, leave the field empty and click **Save**. Pont then uses the environment key, if one is set.

Admins can check a provider via the API: `GET /api/providers/:id/models` lists live models, and `POST /api/providers/:id/test` sends a test chat.

Choose eligible models

The eligible-models list controls which Pont model IDs the gateway accepts and returns from `/v1/models`. **Until you save a list for the first time, only OpenCode Go models are allowed.**

1. Open **Eligible models**. Models are grouped by provider and shown with checkboxes. Only providers that have a key are listed.
2. If you haven't saved a list yet, only OpenCode Go is shown. Click **Show other providers** to add OpenRouter models.
3. Tick the models you want to allow and untick the rest.
4. Click **Save**. The status line then reads *"Using your saved allowlist."* From now on, *only* the ticked models are accepted.

Manage users

Create a user

1. Open **Users**. The table lists each account with name, email, role, status and key prefix.
2. Under **Create user**, fill in *Username*, *Password* (8–200 characters), and optionally *Email*, *First name* and *Last name*. Tick **Admin** only if the user should have administrator rights.
3. Click **Create**. The new user's **gateway key is shown once**. Give it to the user through a secure channel.

Rotate a user's key

1. On **Users**, click **Rotate key** in that user's row.
2. Copy the new key that appears below the table. The old key stops working immediately.

Edit, disable, reset a password, or delete

The console doesn't have controls for these actions. Use the management API instead, either from the Endpoints explorer or with `curl` (see the next chapter):

- **Update or disable:** `PUT /api/users/:username` with *all* fields: `email`, `firstName`, `lastName`, `enabled`, `admin`. Set `"enabled": false` to block both sign-in and gateway access.
- **Reset a password:** use the same call and add `"password"` (8–200 characters). If you leave it out or send an empty value, the password stays the same.
- **Delete:** `DELETE /api/users/:username`.

CHAPTER 07

Using the API

Pont has two APIs: the **gateway**, for LLM calls with your gateway key, and the **management API** under `/api`, which uses a session token.

Gateway API (LLM calls)

BASE URL	BEHAVIOR
`http://127.0.0.1:9880/v1`	**Recommended.** One endpoint for all providers. The `model` must start with `opencode-` or `openrouter-`.
`…/openrouter/v1`	Always uses OpenRouter (`models`, `chat/completions`, `responses`).
`…/opencode-go/v1`	Always uses OpenCode Go (also `messages`).
`…/zen/go/v1`	Legacy OpenCode Go path, kept for existing clients.

Every base URL supports `GET models`, `POST chat/completions` (OpenAI) and `POST responses` (OpenAI Responses). `/v1` and the OpenCode Go routes also support `POST messages` (Anthropic format). Set `"stream": true` to stream the response as server-sent events.

Request headers

HEADER	MEANING
`Authorization: Bearer pont_sk_…`	Required. Your gateway key. Anthropic-style SDKs can send `x-api-key: pont_sk_…` instead.
`x-opencode-session`	Optional conversation ID that Pont passes to the provider. If you don't send one, Pont generates one and returns it in the response header.
`x-yoda-client-app`	Optional app label for logs and stats. You can also send it as `metadata.client_app` in the body.

LIST THE MODELS YOU MAY USE

```
curl http://127.0.0.1:9880/v1/models \
  -H "Authorization: Bearer $PONT_KEY"
```

CHAT COMPLETION (OPENAI FORMAT)

```
curl http://127.0.0.1:9880/v1/chat/completions \
  -H "Authorization: Bearer $PONT_KEY" \
  -H "Content-Type: application/json" \
  -H "x-yoda-client-app: my-script" \
  -d '{
    "model": "opencode—<model-slug>",
    "messages": [{ "role": "user", "content": "Say hi in five words." }]
  }'
```

ANTHROPIC MESSAGES FORMAT

```
curl http://127.0.0.1:9880/v1/messages \
  -H "x-api-key: $PONT_KEY" \
  -H "Content-Type: application/json" \
  -d '{ "model": "opencode—<model-slug>", "max_tokens": 64,
        "messages": [{ "role": "user", "content": "Say hi in five words." }] }'
```

PYTHON, OPENAI SDK

```
from openai import OpenAI

client = OpenAI(base_url="http://127.0.0.1:9880/v1", api_key="pont_sk_…")
reply = client.chat.completions.create(
    model="openrouter—<vendor>-<model>",
    messages=[{"role": "user", "content": "Hello"}],
)
print(reply.choices[0].message.content)
```

AUTOMATIC BRIDGING

Some OpenCode Go models are served only through the Anthropic Messages or Responses API. If you call them with `chat/completions`, Pont converts the request and the response for you, including streaming.

On the pinned routes (`/openrouter/v1`, `/opencode-go/v1`, `/zen/go/v1`), a model name without a Pont prefix is passed to the provider unchanged.

Management API

First sign in to get a session token, then send it as `Authorization: Bearer <token>`. The token is valid for 24 hours.

SIGN IN (POWERSHELL)

```
$body  = @{ username = "warehouse-management"; password = "<your password>" } | ConvertTo-
Json
$login = Invoke-RestMethod -Method Post -Uri http://127.0.0.1:9880/api/login `
           -ContentType "application/json" -Body $body
$h     = @{ Authorization = "Bearer $($login.token)" }
Invoke-RestMethod http://127.0.0.1:9880/api/stats -Headers $h
```

Endpoints for every signed-in user

ROUTE	PURPOSE
`POST /api/login`	Exchanges `{username, password}` for `{token, user}`. No auth needed.
`GET /api/me`	Your account.
`GET /api/me/gateway-key`	Your key prefix (never the full key).
`POST /api/me/gateway-key/rotate`	Creates a new key and returns it once.
`GET /api/stats`	Usage for the last 24 hours plus all-time totals (yours, or all users for admins).
`GET /api/logs?limit=50&offset=0`	Request logs, paged (limit 1–200). Optional `app=` filter.
`GET /api/models`	Model catalog with prices and context length.
`GET /api/vpn/status?refresh=1`	Tunnel state and public egress IP.
`GET /api/events`	Live server-sent event stream of completed requests.
`GET /health`, `GET /health/opencode`	Liveness check, and a reachability check of the OpenCode Go provider. No auth needed.

Admin-only endpoints

ROUTE	PURPOSE
`GET /api/users` · `POST /api/users`	List users, or create one (returns its gateway key once).
`PUT /api/users/:username`	Update a user; all fields are required, `password` is optional.
`DELETE /api/users/:username`	Delete a user.
`POST /api/users/:username/gateway-key/rotate`	Rotate another user's key.

ROUTE	PURPOSE
`GET /api/providers` · `PUT /api/providers/:id`	Provider key status, or set a key with `{"apiKey": "…"}` (empty string clears it).
`GET /api/providers/catalog`	Provider definitions and gateway prefixes.
`GET /api/providers/:id/models` · `POST /api/providers/:id/test`	Live model list, or a test chat (optional `{"model": "…"}`).
`GET /api/eligible-models` · `PUT /api/eligible-models`	Read or save the allowlist: `{"enabled": ["opencode-…", …]}`.

EXAMPLE: DISABLE A USER (ADMIN TOKEN)

```
curl -X PUT http://127.0.0.1:9880/api/users/warehouse-management \
  -H "Authorization: Bearer $ADMIN_TOKEN" -H "Content-Type: application/json" \
  -d '{ "email": "", "firstName": "", "lastName": "", "enabled": false, "admin": false }'
```

CHAPTER 08

Troubleshooting and FAQ

The common errors are listed below with their exact messages, followed by answers to frequent questions.

MESSAGE	CAUSE AND FIX
`401` Invalid or missing Pont API key	The gateway key is missing, wrong or rotated, or the account is disabled. Use your current `pont_sk_…` key, not the session token.
`401` Missing bearer token / Invalid or expired token	A management API call without a valid session. Sign in again, since tokens expire after 24 hours. Disabled accounts also get this error.
`401` Admin access required	The route is admin-only. Ask an administrator.
`401` No OpenRouter API key: set one in the admin panel or OPENROUTER_API_KEY	The provider has no key. An admin needs to add one under *Providers*. OpenCode Go returns the same message with its own name.
`400` Model must start with a provider prefix…	On `/v1`, the model must start with `opencode-` or `openrouter-`. Copy the ID from the *Models* page.
`400` Model is not eligible for this gateway	The model isn't on the allowlist. An admin can enable it under *Eligible models*.
`502` <Provider> is unreachable	Pont couldn't reach the provider. Check your internet or VPN connection on the *Overview* VPN panel.
`503` OPENCODE_GO_API_KEY is not configured	Returned by `/health/opencode` when OpenCode Go has no key, either saved or from the environment.

Q The console says "Invalid username or password".

Check the spelling of the username. Make sure the account is enabled. An admin can reset the password with `PUT /api/users/:username`.

Q The Models page or `/v1/models` is empty.

No provider has a key, or the providers can't be reached. `/v1/models` also hides models that aren't eligible. By default only OpenCode Go models are eligible, so if only OpenRouter has a key, an admin must save an allowlist that includes OpenRouter models.

Q The console doesn't load, or sign-in keeps failing with a network error.

Run `docker compose ps`. Both containers should be *Up*, and the API should be *(healthy)*. Check `docker compose logs pont-api`. If you see *"OpenVPN config missing"*, either add `openvpn/client.ovpn` or set `PONT_OPENVPN_SKIP=1`.

Q I call the API from another web page and get a CORS error.

Browsers may call the API only from `PONT_WEBUI_ORIGIN` (`http://127.0.0.1:9881`). Server-side clients, scripts and SDKs aren't affected.

Q Where are my logs and keys stored? Will a rebuild delete them?

Everything is stored in SQLite on the `pont_pont-data` volume. `docker compose up -d --build` and `docker compose down` keep it. Only `down -v` deletes it.

Q Can I see my full gateway key again?

No. Only a hash is stored. Rotate the key on the *API key* page and update your clients.

Q How do I tell Pont which app is calling?

Send `x-yoda-client-app: <name>`. The names `nanobot`, `open-webui`, `nanobrowser` and similar are grouped as `agent`, `chat` and `extension`.